

ПРАКТИЧЕСКОЕ ЗАДАНИЕ №3

КЛАССИЧЕСКИЙ МЕТОД ПОИСКА КЛЮЧЕВЫХ ТОЧЕК

1. ЦЕЛЬ РАБОТЫ

Познакомиться с классическим методом поиска ключевых точек и их соответствий, освоить применение SuperPoint для поиска ключевых точек и проанализировать полученные в ходе выполнения работы результаты.

2. ОСНОВНЫЕ ТЕОРЕТИЧЕСКИЕ ПОЛОЖЕНИЯ

2.1 Что такое ключевые точки

Ключевые точки — это характерные области изображения, хорошо различимые и устойчивые к изменениям масштаба и освещения. Окрестности ключевых точек уникальны, и служат опорой для анализа, сопоставления и восстановления структуры изображения.

2.2 Зачем нужны ключевые точки

Поиск ключевых точек и их сопоставление с рядом изображений является начальным этапом перехода от двухмерных задач к трехмерным. При помощи данного метода решаются задачи моделирования трехмерного объекта, определение положения объекта в пространстве и вычисление расстояние до него.

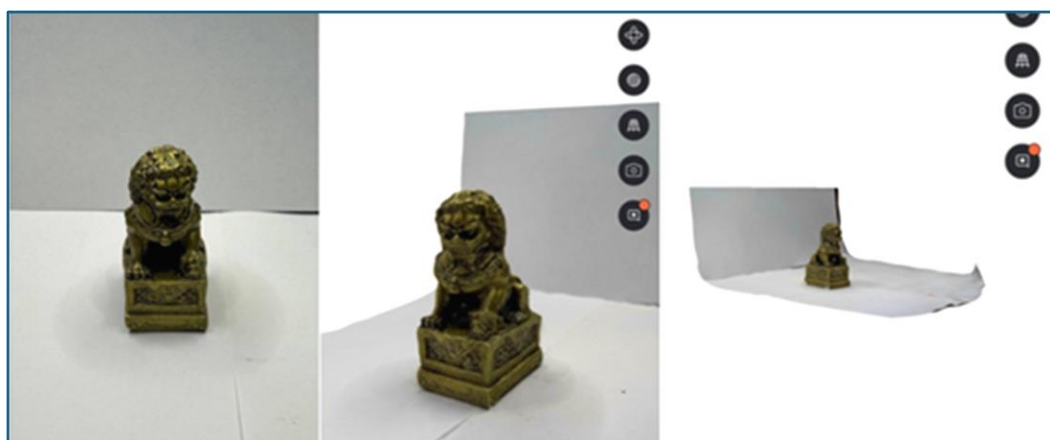


Рисунок 1 – Полученная модель при помощи метода 3D – сканирования с использованием Polycam

Практическое применение данного метода можно найти в таких сферах, как:

- Беспилотные автомобили и транспорт:
 - Понимание сцены (дистанция до машин, пешеходов, разметки);
 - Помощь в торможении, парковке, удержании полосы.
- Дополненная и виртуальная реальность (AR/VR):
 - Реалистичное совмещение виртуальных объектов с реальной сценой;
 - Оклюзия (когда виртуальный объект должен скрываться за реальным).
- Медицина и анализ полученных в ходе обследований изображений:
 - Сегментация и измерение объёмов органов (полученных, к примеру при помощи магнитно-резонансной томографии).

2.2.1 Классический метод поиска ключевых точек

Классический метод поиска ключевых точек делиться на следующие этапы:

1. Детекция ключевых точек и определение их дескрипторов (описание ключевых точек);
2. Сравнения их дескрипторов;
3. Построения линий смещения на изображении.

Алгоритмы детекции ключевых точек называются детекторами. Наиболее часто применяемый из них - детектор углов Харриса (Harris Corner Detector). Данный алгоритм исследует локальные участки изображения и выделяет точки, где интенсивность яркости при смещении участка изменяется в любом направлении. Математическая формулировка описанного определения:

$$E(u, v) = \sum_{x,y} w(x, y) (I(x - u, y + v) - I(x, y))^2, \quad (1)$$

Где:

- $E(u, v)$ – функция изменения локального участка при смещении (u – вдоль оси x , v – вдоль оси y);
- $w(x, y)$ – весовая функция (бинарное окно или функция Гаусса);

- $(I(x - u, y + v) - I(x, y))^2$ – квадрат разницы яркости между смещенным изображением и исходным.

Помимо детектора Харриса, широкую известность приобрел детектор FAST (Features from Accelerated Segment Test). В ходе работы данного детектора производится проверка яркости последовательно расположенных локальных участков вокруг предполагаемой ключевой точки и производится их фильтрация, что обеспечивает его высокую скорость обработки информации.

После обнаружения ключевых точек вычисляются дескрипторы (вектора признаков, описывающие локальную область изображения вокруг ключевых точек). Дескрипторы подразделяются на гистограммные и бинарные.

Гистограммные дескрипторы основываются на направлении градиента яркости в окрестности точки и формируют вектор из вещественных чисел (зачастую являющихся нормализованными). Примеры гистограммных дескрипторов:

- SIFT (Scale-Invariant Feature Transform) — один из первых дескрипторов, устойчивых к изменению масштаба;
- SURF (Speeded-Up Robust Features) — упрощённая и ускоренная версия SIFT.

Бинарный дескриптор использует попарное сравнения интенсивности пикселей. Результатом являются бинарные строки (последовательности, состоящие из нулей и единиц), что ускоряет сопоставление точек. Примеры бинарного дескриптора:

- BRIEF (Binary Robust Independent Elementary Features) — использует случайные пары пикселей для бинарных сравнений;
- BRISK/FREAK — используют круговую(кольцевую) структуру выборки вокруг ключевой точки.

2.3. Нейросетевой подход SuperPoint

SuperPoint — современный нейросетевой алгоритм, предназначенный для одновременного обнаружения (детектирования) и описание (дескрипции) ключевых точек

на изображении. В отличие от SIFT, BRISK и других алгоритмов, которые основаны на ручных эвристиках, обучение SuperPoint производилось с использованием методов глубокого обучения.

Основным элементом сети является кодировщик (Encoder) — набор сверточных слоев, который извлекает из входного изображения высокоуровневые признаки, служащие основой для последующего детектирования ключевых точек и построения их дескрипторов.

Далее архитектура разделяется на два блока из детектора (Detector head) и дескриптора (Descriptor head). Обе эти части работают на одних и тех же признаках, вырабатываемых кодировщиком. При детектировании генерируется карта вероятности, показывающая, насколько каждый пиксель на изображении является ключевой точкой. При описании формируются векторы признаков изображения по каждой ключевой точке.

3 ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ И ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Подготовка данных

- Сделайте две фотографии одной и той же сцены с одним выраженным объектом (архитектурный элемент, предмет интерьера, статуя и т.д.). Убедитесь, что объект четко виден и занимает центральное положение в кадре.
- Запишите два коротких видео (6-10 секунд) с этим объектом. На первом видео не должно быть смещение или изменение кадра, на втором видео должно быть незначительное смещение (движение камеры в сторону).
- Убедитесь, что у вас есть:
 1. Репозиторий [SuperPointPretrainedNetwork](#);
 2. Файл весов superpoint_v1.pth;
 3. Ваше изображение размера не больше 500 Кб и преобразовано в формат png;
 4. Ваше видео преобразовано в формат .avi.

3.2 Алгоритм выполнения работы:

1. Инициализация нейросетевого детектора ключевых точек SuperPoint

Перед началом работы импортируйте библиотеки `cv2`, `numpy`. Добавьте путь к репозиторию `SuperPointPretrainedNetwork` для доступа к модулю `demo_superpoint.py`. Импортируйте класс `SuperPointFrontend` и создайте его экземпляр, указав путь к файлу весов `superpoint_v1.pth`, и установив параметры в зависимости от ваших входных данных:

- `nms_dist` — радиус подавления для уникальных ключевых точек;
- `conf_thresh` — минимальный порог уверенности для фильтрации точек;
- `nn_thresh` — контроль качества сопоставления точек между изображениями.

```
import cv2
import numpy as np
import sys
sys.path.append('C:/CV/SuperPointPretrainedNetwork') # Добавление пути к репозиторию с моделью SuperPoint
from SuperPointPretrainedNetwork.demo_superpoint import SuperPointFrontend

# Инициализация объекта SuperPointFrontend
fe = SuperPointFrontend(weights_path='SuperPointPretrainedNetwork/superpoint_v1.pth',
                        nms_dist=4,
                        conf_thresh=0.015,
                        nn_thresh=0.7
                        )
```

Рисунок 2 – Пример кода

2. Извлечение ключевых точек и дескрипторов

Обработка изображения начинается с извлечения ключевых точек, устойчивых к искажениям. Обработка выполняется при помощи нейросетевой модели SuperPoint.

```
def get_keypoints_and_descriptors(image):
    # Преобразование изображения в формат float32 и нормализация [0, 1]
    img_float32 = np.float32(image) / 255.0

    # Запуск модели SuperPoint
    pts, desc, _ = fe.run(img_float32)
    return pts, desc
```

Рисунок 3 – Пример кода

Где `pts` — координаты найденных точек на изображении, а `desc` — дескрипторы этих точек (векторные признаки для сопоставления).

3. Преобразование найденных двумерных точек

Для корректной работы программы необходимо преобразовать координаты всех найденных ключевых точек из формата массива в объекты OpenCV (`cv2.KeyPoint`), чтобы обеспечить возможность их последующей визуализации и обработки.

```
# Преобразование координат найденных точек в объекты OpenCV
kp1 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts1[0], pts1[1])]
kp2 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts2[0], pts2[1])]
```

Рисунок 4 – Пример кода

4. Сопоставление точек между изображениями

Для поиска совпадающих пар точек на двух изображениях используется классический алгоритм сопоставления дескрипторов `cv2.BFMatcher`, основанный на сравнении их значений по евклидовому расстоянию.

```
def match_keypoints(desc1, desc2):
    # Инициализация BFMatcher
    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

    # Поиск совпадений между дескрипторами
    matches = bf.match(desc1.T, desc2.T)

    # Сортировка совпадений по расстоянию (меньше - лучше)
    return sorted(matches, key=lambda x: x.distance)
```

Рисунок 5 – Пример кода

5. Визуализация совпадений

Для оценки точности сопоставления применяется визуализация линий между совпадающими точками:

```
def draw_matches(img1, kp1, img2, kp2, matches):
    return cv2.drawMatches(
        img1, kp1, img2, kp2,
        matches[:50], None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

Рисунок 6 – Пример кода

6. Обработка фотографий и вызов функции

На данном этапе проводится экспериментальная проверка функционирования детектора ключевых точек SuperPoint на паре изображений. Ожидается, что при сопоставлении двух фотографий, не имеющих взаимного смещения, линии будут иметь строго горизонтальную ориентацию.

```
def testpoint_detector(img1, img2):
    # Перевод входных изображений в оттенки серого для корректной работы детектора
    img1_gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    img2_gray = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    # Получение координат ключевых точек и дескрипторов для каждого изображения
    pts1, desc1 = get_keypoints_and_descriptors(img1_gray)
    pts2, desc2 = get_keypoints_and_descriptors(img2_gray)

    # Преобразование координат найденных точек в объекты OpenCV
    kp1 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts1[0], pts1[1])]
    kp2 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts2[0], pts2[1])]

    # Визуализация совпадений на исходных изображениях и сохранение результата
    matches = match_keypoints(desc1, desc2)
    img_matches = draw_matches(img1, kp1, img2, kp2, matches)
    cv2.imwrite("Test_Matches.png", img_matches)
```

Рисунок 7 – Пример кода

По итогу у нас должны отображаться совпадающие точки между кадрами и сохраняться финальное изображение Test_Matches.png

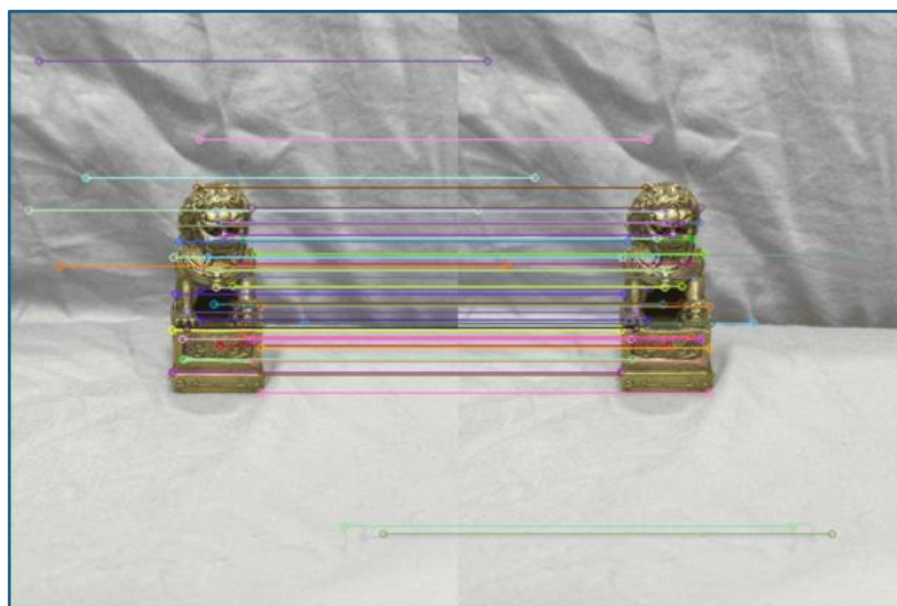


Рисунок 8 – Пример итоговых изображений

7. Обработка видео и вызов функций

Работа ведётся с двумя видеофайлами. Из каждого берётся кадр, выполняется обработка, визуализация выводится на экран, а также сохраняется в выходное видео.


```

cap1 = cv2.VideoCapture(r'C:\Videos\output5.avi')
cap2 = cv2.VideoCapture(r'C:\Videos\output6.avi')
out = None

while True:
    ret1, frame1 = cap1.read()
    ret2, frame2 = cap2.read()
    if not ret1 or not ret2:
        break

    frame1_gray = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    frame2_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    pts1, desc1 = get_keypoints_and_descriptors(frame1_gray)
    pts2, desc2 = get_keypoints_and_descriptors(frame2_gray)

    kp1 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts1[0], pts1[1])]
    kp2 = [cv2.KeyPoint(x, y, 1) for x, y in zip(pts2[0], pts2[1])]

    matches = match_keypoints(desc1, desc2)
    img_matches = draw_matches(frame1, kp1, frame2, kp2, matches)

    if out is None: # Инициализация VideoWriter при первом кадре
        h, w = img_matches.shape[:2]
        fourcc = cv2.VideoWriter_fourcc(*'XVID')
        out = cv2.VideoWriter('matches_output.avi', fourcc, 25, (w, h))

    out.write(img_matches)

    cv2.imshow("Matches", img_matches)
    cv2.imwrite("supperpoint.png", img_matches)

    # Выход из цикла по нажатию 'q'
    key = cv2.waitKey(10) & 0xFF
    if key == ord('q'):
        break

```

Рисунок 9 – Пример кода

В результате должны быть отображены совпадающие точки между кадрами, записываны видео matches_output.avi и сохранено финальное изображение supperpoint.png



Рисунок 10 – Пример итогового изображения

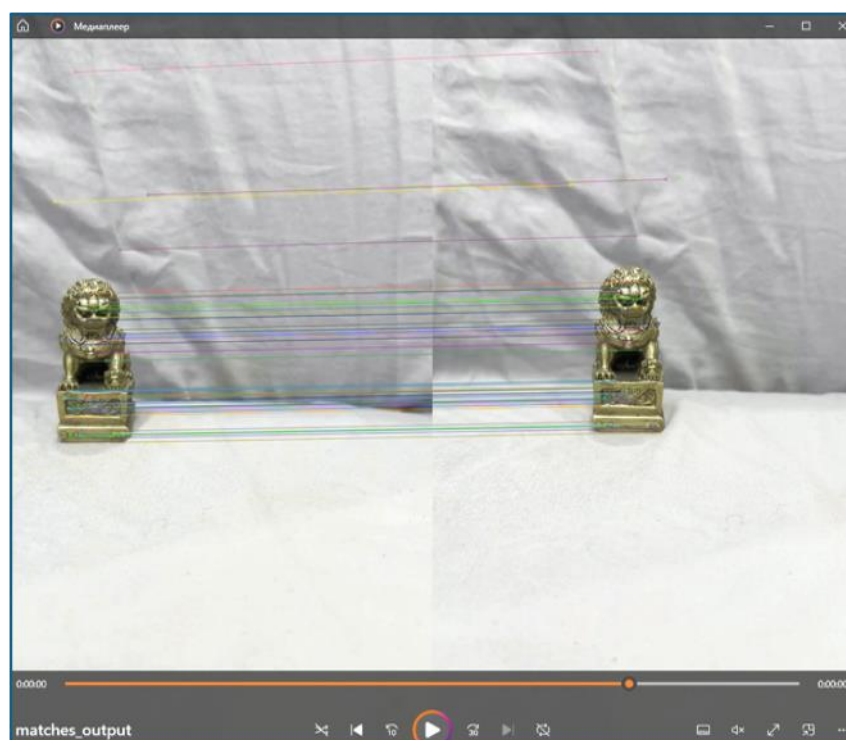
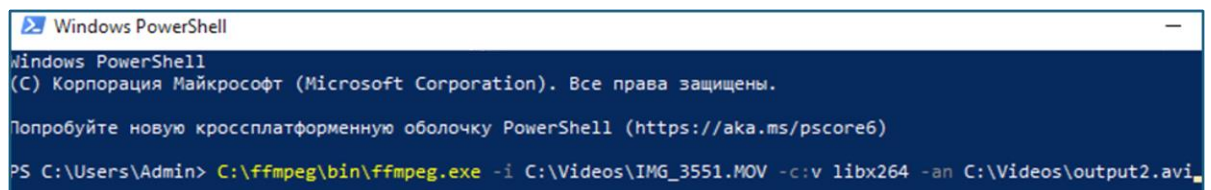


Рисунок 11 – Пример итогового видео

Приложение 1

Пример конвертации видео с использованием FFmpeg.exe в формат .avi в PowerShell с удалением звука:

```
[Путь до файла ffmpeg] -i [Путь исходного файла] -c:v libx264 -an [Полученный файл]
```



```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Попробуйте новую кроссплатформенную оболочку PowerShell (https://aka.ms/pscore6)

PS C:\Users\Admin> C:\ffmpeg\bin\ffmpeg.exe -i C:\Videos\IMG_3551.MOV -c:v libx264 -an C:\Videos\output2.avi
```

Литература:

- Документация по OpenCV
<https://opencv.org/>
- Учебное пособие по калибровке камеры в OpenCV
<https://waksoft.susu.ru/2020/02/29/kalibrovka-kamery-s-ispolzovaniem-s-opencv/>
- Репозиторий и документация SuperPoint
<https://github.com/magicLeap/SuperPointPretrainedNetwork>
- PyTorch Hub — MiDaS (оценка глубины)
https://pytorch.org/hub/intelisl_midas_v2/
- Руководство по работе с MiDaS от Intel
<https://github.com/isl-org/MiDaS>
- Обзор детекторов и дескрипторов в OpenCV (SIFT, ORB, FAST, Harris)
https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html
- Шпаргалка по компьютерному зрению с OpenCV и Python
<https://tproger.ru/translations/opencv-python-guide>
- Модель для оценки глубины
<https://huggingface.co/spaces/depth-anything/Depth-Anything-V2>